pytest 實戰講義

> 7 個單元，每個單元：**先看懂一個範例，再解一個實務情境題** 實務題全部取材自資安監控平台（SOC）的真實工作 —— 也就是啄木科技 XVR 在做的事

開始之前

這份講義怎麼用

每個單元都是同樣的節奏：

> ① 一句話重點 → ② 為什麼需要 → ③ 怎麼寫（看範例） → ④ 常見錯誤 → ⑤ 實務練習

範例用一個簡單的購物車，目的是讓你看懂語法。 **實務練習**換成資安情境，目的是讓你練習「判斷該測什麼」—— 這才是測試工程師真正的工作。

資料夾

```
pytest-練習/
├── 01_pytest講義.pdf          ← 你在看的
├── 02_答案與預期輸出.pdf      ← 對答案用
├── example/                   ← 範例（filled_cart），直接跑
│   ├── cart.py
│   └── step1 ~ step7
├── practice/                  ← 實務練習，你來寫
│   ├── xvr_lite.py            ← 被測試的系統，不要改
│   ├── conftest.py
│   └── test_u1 ~ test_u7
└── answer/                    ← 參考解答
```

安裝

```
pip install pytest
```

實務練習的主角：XVR-lite

`practice/xvr_lite.py` 是一個縮小版的資安監控管線，流程跟真實 SOC 平台一樣：

```
原始日誌 ──► 解析 ──► 偵測規則 ──► 告警 ──► 自動回應 ──► 防火牆
            Unit 1-3   Unit 4-6          Unit 7
```

元件	做什麼	在真實的 XVR 裡
`parse_event`	把一行 JSON 日誌轉成標準格式	接入端點、防火牆、DNS 等各種日誌來源
`ingest_batch`	批次匯入，跳過壞資料	客戶環境的日誌品質參差不齊
`BruteForceRule`	5 分鐘內同一 IP 登入失敗 5 次 → 告警	偵測規則，對應 MITRE ATT&CK T1110
`Responder`	決定要不要自動封鎖	主動防禦、自動聯防

開始寫練習之前，先花 5 分鐘讀一遍 `xvr_lite.py`。每個函式的 docstring 就是規格，後面所有題目的「正確答案」都以它為準。

Unit 1 | 第一個測試

① 一句話重點

> **測試就是一個以 `test_` 開頭的函式，裡面用 `assert` 寫下你的期望。**

② 為什麼需要

想像你改了一行程式，然後手動開程式、輸入資料、看結果對不對。改十次就要做十次。

測試就是把「手動檢查」寫成程式，之後一個指令就能重跑全部。

③ 怎麼寫

```
# example/step1_first_test.py
from cart import ShoppingCart

def test_total_is_price_times_qty():
    cart = ShoppingCart()           # 準備
    cart.add("蘋果", price=30, qty=3)  # 執行
    assert cart.total() == 90       # 驗證
```

這三行有個固定的模式，叫 **AAA**：

	英文	做什麼
準備	Arrange	建立要測的東西

	英文	做什麼
執行	Act	呼叫你要測的功能
驗證	Assert	檢查結果

幾乎所有測試都長這樣。寫不出來的時候，先問自己這三格各要填什麼。

pytest 怎麼找到你的測試？ 靠名字：

	規則
檔名	`test_*.py` 或 `*_test.py`
函式	`test_` 開頭
類別	`Test` 開頭，而且不能有 `__init__`

```
cd example
pytest stepl_first_test.py       # 跑一個檔案
pytest -v stepl_first_test.py    # -v 列出每個測試的名字
```

④ 常見錯誤

- **函式忘了 `test_` 開頭** → pytest 直接無視它，你還以為測試過了
- **檢查 None 用 `== None`** → 要用 `is None`
- **一個測試塞太多檢查** → 失敗時看不出是哪個功能壞了。一個測試驗證一件事

⑤ 實務練習：新日誌來源上線前的驗證

> 📁 `practice/test_ul_parse.py`

情境 客戶的 SOC 要接一台新的防火牆。開發同事寫好了日誌解析器 `parse_event()`， 接進正式環境之前，主管請你先確認「正常的日誌」都能被正確解析。

任務：寫 5 個測試 —— 基本欄位、時間轉型、缺少選填欄位、port 型別、IPv6。

驗收：`5 passed`

提示：先讀 `parse_event` 的 docstring，它寫了每個欄位解析後應該是什麼型別。

> 💼 **為什麼這題跟啄木有關** 啄木的 Detection Engineering 職缺寫著「分析新接入的日誌來源，學習判讀欄位與事件內容」。 每接一種新設備，第一件事就是確認解析正確 —— 解析錯了，後面所有偵測規則都是白做。

Unit 2｜讀懂失敗訊息

① 一句話重點

> **測試失敗時，pytest 會告訴你「預期是什麼、實際是什麼」。先學會讀它，再學會判斷是誰錯。**

② 為什麼需要

測試工程師每天花最多時間的事，不是寫測試，是**看測試為什麼失敗**。

而且測試失敗不一定代表程式有問題 —— 也可能是測試本身寫錯了。分辨這兩種，是基本功。

③ 怎麼讀

```
pytest step2_reading_failures.py
```

```
    def test_wrong_number_on_purpose():
        cart = ShoppingCart()
        cart.add("蘋果", price=30, qty=2)
>       assert cart.total() == 100
E       assert 60 == 100
E        +  where 60 = total()
```

三個地方要看：

符號	意思
`>`	失敗的是哪一行
`E`	錯誤說明。`assert 60 == 100` → 實際是 60，你期望 100
`where`	pytest 幫你把 `cart.total()` 算出來了，不用自己 print

這個功能叫 **assertion rewriting**，是 pytest 比內建 unittest 好用的主要原因之一。

最下面還有一行總結：

```
========================= 2 failed, 1 passed in 0.15s ==========================
```

判斷是誰錯：兩種可能

	測試寫錯了	產品有 bug
意思	期望值跟規格不符	程式行為跟規格不符

	測試寫錯了	產品有 bug
處理	修改測試	開 bug 回報，**不要改測試去配合**

判斷的依據只有一個：規格。 不是你的直覺，也不是「程式現在怎麼跑」。

④ 常見錯誤

- **看到紅燈就改測試讓它變綠** → 這是最危險的習慣。你可能剛好把一個真 bug 藏起來了
- **只看最後一行總結** → 往上捲，`E` 開頭那幾行才是重點

> 💡 實用指令：`pytest -x` 第一個失敗就停下；`pytest --lf` 只重跑上次失敗的

⑤ 實務練習：CI 亮紅燈，你負責分診

> 📁 `practice/test_u2_triage.py`

情境 早上 9 點，CI 通知解析器的測試失敗了。這四個測試是前一位同事寫的，有兩個是紅的。

任務 1. 跑測試，讀懂每一個失敗訊息 2. 對照 `parse_event` 的規格，判斷每個失敗是「測試寫錯」還是「產品有 bug」 3. 測試寫錯的 → 修測試 4. 產品有 bug 的 → **不要改程式、不要改測試**，在檔案底部寫一份 bug 回報

驗收：`3 passed, 1 failed`，而且 bug 回報寫得出「重現步驟、預期結果、實際結果、影響範圍」

提示：其中一個失敗訊息是 `assert '443' == 443`。注意引號。

> 💼 **為什麼這題跟啄木有關** 職缺寫著「執行測試、缺陷追蹤、產出測試報告」。分診和寫 bug 回報就是這件事。 面試時你可以說：「我的原則是先對照規格判斷是誰錯，絕不為了讓 CI 變綠去改測試。」

Unit 3 | 測試「應該要出錯」的情況

① 一句話重點

> **用 `pytest.raises` 確認程式在遇到壞資料時，丟出「正確的」例外。**

② 為什麼需要

只測正常情況，就像只檢查門鎖用鑰匙打得開，卻沒檢查用鐵絲打不打得開。

好的測試要確認兩件事： 1. 壞資料**會被擋下來** 2. 擋下來的**理由是對的**

③ 怎麼寫

```
# example/step3_exceptions.py
import pytest

def test_negative_price_raises():
    cart = ShoppingCart()
    with pytest.raises(ValueError):       # 「我預期這裡會丟 ValueError」
        cart.add("蘋果", price=-1)
```

如果 `with` 區塊裡**沒有**丟出 ValueError，測試就會失敗。

連訊息一起檢查：

```
with pytest.raises(ValueError, match="價格"):
    cart.add("蘋果", price=-1)
```

為什麼要檢查訊息？因為購物車有兩條規則都丟 `ValueError`（價格為負、數量為零）。只檢查例外類型的話，你分不出是哪條規則被觸發 —— 測試可能因為「錯的理由」而通過。

`match` 用的是正規表示式，會在錯誤訊息裡搜尋。

④ 常見錯誤

- **直接呼叫會丟例外的函式，沒包** `pytest.raises` → 例外往外炸，測試變成 ERROR
- **用 try/except 自己接** → 如果程式根本沒丟例外，try/except 版本會默默通過。`pytest.raises` 不會
- **例外類型抓太寬**，例如 `pytest.raises(Exception)` → 什麼錯都算通過，等於沒測

⑤ 實務練習：客戶環境的髒資料

📁 `practice/test_u3_malformed.py`

情境 XVR 是地端部署產品，裝在客戶的機房裡。客戶的設備五花八門，送進來的日誌常常殘缺不全。上週有客戶回報：某台舊設備送了一行亂碼，結果**整批日誌都沒進系統**。團隊加強了防護，你要寫測試確保它真的有效。

任務 - `parse_event` 遇到 5 種壞資料（非 JSON、缺欄位、IP 錯、port 超範圍、未知 action）都要丟 `MalformedEvent`，而且訊息說得出原因 - `ingest_batch` 遇到壞資料要跳過並記錄「第幾行」，不能整批停擺

驗收：`7 passed`

提示：最後兩題是 `ingest_batch`，它**不應該**丟例外。這種題目不需要 `pytest.raises` —— 測試本身能跑完，就代表沒丟例外。

> 💼 **為什麼這題跟啄木有關** 地端部署代表你無法控制輸入。一行壞資料讓整個平台停擺，客戶在那段時間就完全沒有防護。「異常輸入不能讓系統崩潰」是資安產品最基本的韌性要求。

Unit 4 | fixture：每個測試都要乾淨的起點

① 一句話重點

> **把「每個測試開頭都要做的準備」抽出來變成 fixture，pytest 會自動幫你準備好、用完自動收拾。**

② 為什麼需要

兩個問題：

問題一：重複。 五個測試開頭都在寫一樣的四行準備程式碼。改一個地方要改五次。

問題二：互相汙染。 如果測試 A 留下了資料，測試 B 可能因此莫名其妙地失敗或通過。 測試之間必須互相獨立，套件才能被信任。

③ 怎麼寫

```
# example/step4_fixture.py
import pytest

@pytest.fixture
def filled_cart():
    cart = ShoppingCart()
    cart.add("蘋果", price=30, qty=2)
    return cart

def test_new_total(filled_cart):       # 參數名稱跟 fixture 一樣，pytest 就會自動傳進來
    assert filled_cart.total() == 60
```

需要收尾的時候，把 `return` 換成 `yield`：

```
@pytest.fixture
def db_connection():
    conn = connect()      # yield 之前：準備
    yield conn            # 測試在這裡執行
    conn.close()          # yield 之後：收拾（即使測試失敗也會執行）
```

放在 `conftest.py`，所有測試檔都能用，不需要 import。pytest 會自動載入它。

scope：多久建立一次

scope	建立時機	適合
`function`（預設）	每個測試都重建	有狀態的東西 —— 大部分情況用這個
`module`	每個檔案一次	
`session`	整輪測試只一次	**起 Docker 容器、連資料庫**這種很慢的資源

`session` 比較快，但代價是測試之間共用同一份資源，可能互相影響。

進階：工廠型 fixture

有時候測試需要「很多個類似但不同的東西」。這時候讓 fixture 回傳一個**函式**：

```
@pytest.fixture
def make_item():
    def _make(name, price=10):
        return {"name": name, "price": price}
    return _make

def test_xxx(make_item):
    a = make_item("蘋果")
    b = make_item("西瓜", price=200)
```

④ 常見錯誤

- **參數名稱打錯** → `fixture 'xxx' not found`
- **為了快而亂用 `session` scope** → 測試開始互相影響，而且只在某些執行順序下才失敗，非常難查
- **在 fixture 裡寫 assert** → fixture 是準備資源用的，驗證放在測試裡

> 💡 一定要試：把範例的 `@pytest.fixture` 改成 `@pytest.fixture(scope="module")` 跑一次，看 `test_still_get_clean_cart` 怎麼壞掉。這個體驗比讀十遍說明有用。

⑤ 實務練習：測試一條有「記憶」的偵測規則

> 📁 `practice/test_u4_fixture.py` + `practice/conftest.py`

情境 Detection Engineering 團隊寫好一條暴力破解偵測規則（MITRE ATT&CK T1110）：

> **同一個來源 IP，5 分鐘內登入失敗 5 次 → 告警**

這條規則跟前面不同 —— 它會**記住**每個 IP 失敗過幾次。 如果上一個測試留下了失敗紀錄，下一個測試就可能被誤觸發。

任務 1. 在 `conftest.py` 寫兩個 fixture： - `engine`：乾淨的偵測引擎，測試結束後 `reset()` - `make_event`：工廠型 fixture，快速產生事件 2. 寫 6 個測試：4 次不告警、5 次告警、引擎每次都乾淨、不同 IP 分開算、成功登入清除紀錄、告警後不洗版

驗收：`6 passed`

提示：第 3 題 `test_engine_is_fresh_each_test` 排在「產生告警」的測試之後。如果你的 fixture 寫錯了，它就會失敗。

> 💼 **為什麼這題跟啄木有關** 偵測規則幾乎都有狀態：時間窗、計數、基線。這類規則的測試最容易互相汙染。測試做「登入成功清除紀錄」那題時，想想看這條規則有什麼漏洞 —— 那正是 Detection Engineering 每天在想的事。

Unit 5 | parametrize：一次測遍所有邊界

① 一句話重點

> **同一段測試邏輯，用一張表列出多組輸入和期望結果，pytest 會把每一組變成獨立的測試。**

② 為什麼需要

測試一個門檻規則，你需要測很多個數字。寫成五個幾乎一樣的函式很浪費， 改用 for 迴圈又有個問題：**第一組失敗就停了，後面根本沒跑**。

parametrize 解決這兩件事。

③ 怎麼寫

```
# example/step5_parametrize.py
@pytest.mark.parametrize("code, expected_price", [
    ("SAVE10", 90),
    ("SAVE20", 80),
    ("HALF",   50),
])
def test_new_discount_codes(code, expected_price):
    cart = ShoppingCart()
    cart.add("蘋果", price=100)
    assert cart.total(code) == expected_price
```

跑 `-v` 會看到三個獨立的測試：

```
test_new_discount_codes[SAVE10-90] PASSED
test_new_discount_codes[SAVE20-80] PASSED
test_new_discount_codes[HALF-50] PASSED
```

用 `ids=["10_percent_off", "20_percent_off", "half_price"]` 可以替每組取名字，報告更好讀。

要測哪些值？兩個基本技巧

這是測試設計的核心，面試問「你怎麼設計測試案例」就是在問這個。

等價類劃分：把輸入分成「行為一樣」的幾群，每群挑一個代表就好。

> 規則「數量必須大於 0」→ 分成兩群：{..., -1, 0}（不合法）和 {1, 2, ...}（合法） 合法那群裡 5 和 500 行為一樣，不用都測

邊界值分析：規則的交界處最容易寫錯，所以每個邊界的**前後**都要測。

> 規則「數量必須大於 0」→ 邊界在 0 和 1 之間 → 測 0 和 1 寫成 `qty < 0` 或 `qty <= 0`，差別只在 0 這個點

為什麼	例子
工程師最常在 `<` 和 `<=` 之間寫錯	只測 5 和 500，抓不到 `<=` 寫成 `<`

④ 常見錯誤

- **只測「中間」的值** → 邊界的 bug 全部漏掉
- **參數名稱跟函式參數對不上** → `"a, b"` 字串裡的名字要和函式的參數一模一樣
- **在測試裡寫 for 迴圈取代 parametrize** → 一個失敗就看不到其他組的結果

⑤ 實務練習：誤報與漏報的分界線

> 📁 `practice/test_u5_boundary.py`

情境 兩個客戶，兩種抱怨：

> 客戶 A：「每天一堆暴力破解告警，點進去都只是員工忘記密碼。」 → **誤報** 客戶 B：「我們被打了，你們的系統卻沒叫。」 → **漏報**

兩種抱怨都指向同一件事：**門檻和時間窗**。門檻太低會誤報、太高會漏報。 任何一個 `<` 寫成 `<=`，都會直接反映在客戶的告警量上。

主管要你用 parametrize 把這條規則的邊界全部釘死，之後有人調整門檻，測試會立刻告訴他影響了什麼。

任務：把 4 個測試改寫成 parametrize - 失敗次數門檻：3、4、5、6、10 次 - 時間窗邊界：5 次失敗分散在 240、299、300、301、600 秒內 - 嚴重度：不同帳號對應 high / medium（有一個陷阱） - 不計數的 action（等價類）

驗收：`19 passed`

提示：規格寫「最新事件與最舊事件相差剛好 window 秒，仍算在窗內」。300 和 301 誰會告警？

> 💼 **為什麼這題跟啄木有關** 啄木的新竹市 SBIR 研發計畫主題就是「降低資安誤報率」。 誤報與漏報的分界就是這些邊界值。面試時能用這題解釋「為什麼要測邊界」，會非常有說服力。

Unit 6 | marker：分類、篩選、追蹤已知問題

① 一句話重點

> **替測試貼標籤，就能挑著跑；用 `xfail` 把已知但還沒修的問題留在套件裡追蹤。**

② 為什麼需要

測試越寫越多，會遇到三個問題：

1. 有些測試很慢，開發時不想每次都等
2. 有些測試只在特定環境有意義
3. 發現了 bug 但還沒時間修，測試會一直紅 —— 可是刪掉又怕忘記

③ 怎麼寫

自訂標籤（要先在 `conftest.py` 註冊）：

```
# conftest.py
def pytest_configure(config):
    config.addinivalue_line("markers", "slow: 跑比較久的測試")
```

```
@pytest.mark.slow
def test_large_replay(): ...
```

```
pytest -m "slow"          # 只跑慢的
pytest -m "not slow"      # 跳過慢的（開發時用這個）
```

內建標籤：

```
@pytest.mark.skip(reason="功能還沒做")
@pytest.mark.skipif(sys.platform == "win32", reason="只在 Linux 有意義")
@pytest.mark.xfail(reason="已知缺陷，排在下個版本修")
```

⭐ skip 和 xfail 差在哪

	會不會跑	適合
`skip`	**不跑**	環境不符、功能還沒做
`xfail`	**會跑**，失敗不算錯	**已知缺陷**

已知缺陷為什麼用 `xfail` 不用 `skip`？因為 `xfail` 會持續執行。 哪天有人把 bug 修好了，它會變成 **XPASS**，提醒你「可以把標記拿掉了」。 用 `skip` 的話，你永遠不會知道它已經好了。

輸出符號

符號	意思
`.` / PASSED	通過
`F` / FAILED	失敗
`E` / ERROR	測試本身出錯（例如 fixture 找不到）
`s` / SKIPPED	跳過
`x` / XFAIL	預期中的失敗
`X` / XPASS	預期會失敗，結果居然通過

④ 常見錯誤

- **自訂 marker 沒註冊** → 會跳 `PytestUnknownMarkWarning`
- **用 `xfail` 不寫 reason** → 三個月後沒人記得為什麼
- **把所有失敗的測試都標 xfail** → 只有「確定是已知缺陷、有排修」的才標

⑤ 實務練習：告警品質回歸套件

📁 `practice/test_u6_markers.py`

情境 資安分析師回報兩個問題，都還沒修：

> ① **誤報**：客戶的備份服務帳號 `svc_backup` 每晚重試密碼，每晚都觸發暴力破解告警 ② **漏報**：攻擊者用 5 個不同 IP 各試一次 admin 密碼（分散式暴力破解），規則完全沒偵測到

主管要求：這兩個問題要留在測試套件裡持續追蹤，但不能讓 CI 一直紅。 另外有一個很慢的效能測試，開發時要能跳過。

任務：測試都寫好了，你要**加上正確的 marker** 1. 在 `conftest.py` 註冊 `detection` 和 `slow` 2. 效能測試標 `slow` 3. 誤報和漏報各標 `xfail`，reason 寫清楚 4. Linux 專用的測試用 `skipif` 5. 偵測相關的測試標 `detection`

驗收

```
pytest -v test_u6_markers.py              → 3 passed, 1 skipped, 2 xfailed
pytest -v -m "not slow" test_u6_markers.py → 多出 1 deselected
```

> 💼 **為什麼這題跟啄木有關** 「誤報」和「漏報」是資安偵測產品的兩大品質指標。把已知案例變成測試持續追蹤， 規則每次調整都能立刻知道：修好了哪個、有沒有弄壞別的。這就是「回歸測試」。

Unit 7 | 測試外部依賴：不能真的去封鎖

① 一句話重點

> **程式要呼叫外部系統時，測試裡換成一個「假的」，就能安全地測各種狀況，包括外部系統壞掉。**

② 為什麼需要

如果測試直接呼叫真實的外部系統：

問題	例子
會產生真實後果	真的扣錢、真的封鎖 IP
不穩定	外部系統掛了，你的測試跟著紅
沒網路就不能跑	
測不到異常情況	你沒辦法叫真的防火牆「現在給我逾時」

最後一點最致命：**錯誤處理是最需要測、卻最難在真實環境重現的東西。**

③ 怎麼寫

如果程式設計成「外部依賴可以從外面傳進來」（依賴注入），測試時塞一個假的就好：

```
# example/step7_external_deps.py
class SuccessfulPayment:
    def __init__(self):
        self.charged_amounts = []           # 記錄被呼叫的情況

    def charge(self, amount):
        self.charged_amounts.append(amount)
        return {"success": True, "transaction_id": "TX123"}

def test_checkout_sends_correct_amount():
    cart = ShoppingCart()
    cart.add("蘋果", price=100, qty=2)
    payment = SuccessfulPayment()
    cart.checkout(payment)              # 把假的傳進去
    assert payment.charged_amounts == [200]
```

⭐ 不只檢查結果，還要檢查「互動」

假物件除了回傳資料，還要**記錄自己被怎麼呼叫**。這樣你才能驗證：

- 有沒有被呼叫？呼叫了幾次？
- 傳進去的參數對不對？
- **該不呼叫的時候，有沒有真的不呼叫？** ← 最常被忽略

另一種做法：monkeypatch

如果程式沒有設計成可以注入，就用 pytest 內建的 `monkeypatch` 直接替換：

```
def test_xxx(monkeypatch):
    monkeypatch.setattr(cart, "total", lambda discount_code=None: 999)
```

測試結束後自動還原。

	優點	缺點
依賴注入	穩定、意圖清楚	程式要本來就設計好
monkeypatch	什麼都能換	綁死內部實作，重構就壞

能用注入就用注入。可以記一句話：**好測的程式，通常也是設計得好的程式。**

④ 常見錯誤

- **只檢查回傳值，沒檢查假物件有沒有被呼叫** → 程式可能根本沒呼叫外部系統，直接回傳假資料

- **假物件只做「成功」版本** → 失敗和逾時的處理完全沒測到
- **測試裡不小心用到真的客戶端** → 這題的 `FirewallClient` 被呼叫會直接報錯，就是為了防這個

⑤ 實務練習：自動封鎖，但絕對不能封錯

> 📁 `practice/test_u7_responder.py`

情境 XVR 的賣點之一是**主動防禦**：偵測到高風險攻擊，自動叫防火牆封鎖來源 IP。 這很強，但也很危險 —— 封錯一個 IP，客戶的服務就可能斷線。

所以 `Responder` 刻意把**判斷**和**執行**分開：它只決定要不要封，真正去封的是防火牆。 你要驗證這個判斷完全正確。

任務 1. 寫一個假防火牆：能記錄呼叫，並模擬「成功 / 回傳失敗 / 丟例外」三種狀況 2. 寫 7 個測試： - 高風險 → 封鎖，而且 reason 帶 MITRE 編號 - 中風險 → 只通知，**防火牆一次都沒被呼叫** - 白名單 IP → 不封鎖 - 內網 IP → 不封鎖（用 parametrize 測三個網段） - 防火牆丟例外 → 系統不能崩潰，告警轉人工 - 防火牆回傳失敗 → 告警轉人工 - **端到端**：原始日誌 → 解析 → 偵測 → 回應，整條串起來

驗收：`9 passed`

提示：「防火牆丟例外」那題的測試裡**不應該**出現 `pytest.raises` —— 因為規格要求例外不能往外丟。

> 💼 **為什麼這題跟啄木有關** 啄木官網寫「即時阻斷惡意程式、隔離主機、封鎖端點或中斷網路連線，並透過防火牆等邊界防護協同運作」。 自動化動作是產品最有價值、也最危險的部分，必須用這種方式驗證。 這也呼應你 Threat Intel Agent 的設計：判斷與執行分離，避免誤判直接造成服務中斷。

附錄 A｜常用指令

```
pytest                      # 跑全部
pytest -v                   # 列出每個測試名稱
pytest -s                   # 顯示 print 的內容
pytest -x                   # 第一個失敗就停
pytest --lf                 # 只跑上次失敗的
pytest -k "邊界"            # 名稱包含「邊界」的測試
pytest -m "not slow"        # 依 marker 篩選
pytest 檔案.py::測試名稱     # 只跑一個測試
```

附錄 B｜做完之後的自我檢查

不看講義，能回答這些就代表你真的懂了：

- [ ] 一個測試的 AAA 是哪三格？
- [ ] 測試失敗時，怎麼判斷是測試錯還是產品錯？
- [ ] 為什麼 `pytest.raises` 要加 `match`？
- [ ] fixture 的 `yield` 前後各做什麼？為什麼起容器要用 `session` scope？
- [ ] 什麼是等價類劃分？什麼是邊界值分析？各舉一個例子
- [ ] `skip` 和 `xfail` 差在哪？什麼是 XPASS？
- [ ] 為什麼測試不能呼叫真實的外部系統？假物件除了回傳資料還要做什麼？
- [ ] 依賴注入和 monkeypatch 哪個好？為什麼？

下一步

做完 7 個單元，你已經有能力替自己的專案寫測試了。建議從 **Threat Intel Agent** 開始： 它的結構跟 Unit 7 幾乎一樣 —— 把 VirusTotal、AbuseIPDB 換成假的，測快取、測重試、測風險等級。